

ID	Category	Prompt	Good Response	Bad Response	Good Score	Bad Score	Annotation
FQ-001	Factual Q&A	What is the speed of light in a vacuum?	The speed of light in a vacuum is approximately 299,792,458 meters per second (about 3×10^8 m/s), often denoted as 'c'. This is a fundamental physical constant and, according to special relativity, the maximum speed at which information or matter can travel.	Light travels at around 300,000 km per hour, which is really fast. It's one of the fastest things in the universe.	9	2	Good response: accurate value with correct units (m/s not km/h), includes scientific notation, mentions 'c' notation, and connects to special relativity for depth. Bad response: critically wrong units (km/hour vs km/second — off by factor of 3600), vague phrasing, no scientific context.
FQ-002	Factual Q&A	Who developed the theory of general relativity and when?	Albert Einstein developed the theory of general relativity. He published his field equations in November 1915, completing the theory after roughly a decade of work. General relativity extended his earlier special relativity (1905) and reinterpreted gravity not as a force but as a curvature of spacetime caused by mass and energy.	Einstein came up with relativity in the early 1900s. It was a revolutionary idea about space and time.	9	3	Good response: correct person, precise date (November 1915), distinguishes general from special relativity, explains the core concept. Bad response: vague date ('early 1900s'), no distinction between special and general relativity, no actual explanation of what the theory says.
FQ-003	Factual Q&A	What is the powerhouse of the cell?	The mitochondrion (plural: mitochondria) is known as the powerhouse of the cell. It generates most of the cell's supply of adenosine triphosphate (ATP), the molecule used as chemical energy currency, through a process called cellular respiration (specifically oxidative phosphorylation in the inner mitochondrial membrane).	The nucleus is the powerhouse of the cell because it controls all cell activities.	9	1	Good response: correct organelle, explains ATP, names the exact process (oxidative phosphorylation) and location. Bad response: factually wrong — confuses mitochondria with nucleus. The nucleus is the control center, not energy producer. Critical hallucination-style error.
FQ-004	Factual Q&A	What causes the seasons on Earth?	Earth's seasons are caused by the tilt of its rotational axis (approximately 23.5 degrees), not by its distance from the Sun. As Earth orbits the Sun, this axial tilt means different hemispheres receive sunlight at different angles and for different durations throughout the year. When the Northern Hemisphere tilts toward the Sun, it experiences summer; when it tilts away, it experiences winter.	Seasons are caused by Earth moving closer to and farther from the Sun during its orbit around the Sun.	9	1	Good response: correctly identifies axial tilt (23.5°) as cause, explicitly debunks the common misconception about distance, explains both angle and duration of sunlight. Bad response: states the classic misconception (distance from Sun) — factually incorrect. Earth is actually closest to the Sun in January (Northern Hemisphere winter).
FQ-005	Factual Q&A	What is the largest organ in the human body?	The skin is the largest organ of the human body. In adults, it covers approximately 1.5 to 2 square meters and weighs around 3–4 kg. It serves multiple functions including protecting against pathogens, regulating body temperature, and providing sensory information.	The liver is the largest organ in the human body.	9	1	Good response: correct answer (skin), provides quantitative data (area and weight), explains function. Bad response: factually wrong — the liver is the largest internal organ, but skin is the largest organ overall. A subtle but important distinction the evaluator must catch.
FQ-006	Factual Q&A	How many bones does an adult human body have?	An adult human body has 206 bones. Interestingly, babies are born with around 270–300 bones, but many of these fuse together during childhood and adolescence. By early adulthood, the count stabilizes at 206.	Humans have about 300 bones in their body.	9	3	Good response: correct adult count (206), adds developmental context about infant bones fusing — which is accurate and informative. Bad response: gives the infant count as the adult count, showing no distinction between developmental stages.
FQ-007	Factual Q&A	What is the capital of Australia?	The capital of Australia is Canberra. It was purpose-built as a compromise after Sydney and Melbourne both wanted to be the capital. Located in the Australian Capital Territory (ACT), Canberra has been the seat of the Australian government since 1913.	The capital of Australia is Sydney, as it is the largest and most famous city in the country.	9	1	Good response: correct answer (Canberra), adds historical context about why it was chosen, includes ACT detail. Bad response: states Sydney, a very common misconception — factually wrong. The bad response even justifies the wrong answer with faulty reasoning ('largest city = capital').
FQ-008	Factual Q&A	What is DNA and what does it do?	DNA (deoxyribonucleic acid) is a double-stranded molecule that carries the genetic instructions for the development, functioning, growth, and reproduction of all known organisms and many viruses. It is made of nucleotides containing one of four bases: adenine (A), thymine (T), cytosine (C), and guanine (G). DNA is organized into chromosomes found in the cell nucleus and encodes genes, which are instructions for producing proteins.	DNA is the stuff in your cells that makes you who you are. It contains your genetic code passed down from your parents.	8	3	Good response: full name expansion, structure (double-stranded, nucleotides, four bases), location (nucleus, chromosomes), function (protein encoding). Bad response: technically not wrong but extremely vague — 'stuff in your cells' is not useful, no mechanism or structure explained. Appropriate for a 5-year-old, not an informative answer.
FQ-009	Factual Q&A	What year did World War II end?	World War II ended in 1945. Victory in Europe (V-E Day) was declared on May 8, 1945, following Germany's unconditional surrender. The war in the Pacific ended on September 2, 1945 (V-J Day), when Japan formally surrendered aboard the USS Missouri following the atomic bombings of Hiroshima and Nagasaki.	World War II ended in 1945 when the Allies won.	9	4	Good response: correct year, distinguishes two end dates (V-E Day and V-J Day with exact dates), mentions USS Missouri, explains cause of Japan's surrender. Bad response: correct year but adds nothing — 'Allies won' is not informative and ignores the nuance of two separate surrender events.
FQ-010	Factual Q&A	What is the Pythagorean theorem?	The Pythagorean theorem states that in a right-angled triangle, the square of the length of the hypotenuse (the side opposite the right angle) equals the sum of the squares of the other two sides. Written as a formula: $a^2 + b^2 = c^2$, where c is the hypotenuse and a, b are the other two legs. For example, a triangle with sides 3, 4, and 5 satisfies $9 + 16 = 25$.	The Pythagorean theorem is $a^2 + b^2 = c^2$. It's a famous math formula.	9	3	Good response: defines hypotenuse, explains what each variable represents, gives a concrete numerical example (3-4-5 triple). Bad response: gives the formula but with no explanation of variables, no definition of hypotenuse, no example — not useful to someone who doesn't already know it.
FQ-011	Factual Q&A	What is photosynthesis?	Photosynthesis is the process by which plants, algae, and some bacteria convert light energy (usually from the Sun) into chemical energy stored in glucose. The overall equation is: $6\text{CO}_2 + 6\text{H}_2\text{O} + \text{light energy} \rightarrow \text{C}_6\text{H}_{12}\text{O}_6 + 6\text{O}_2$. It occurs in the chloroplasts of plant cells, where chlorophyll absorbs light. Photosynthesis has two main stages: the light-dependent reactions and the Calvin cycle.	Photosynthesis is how plants make their own food using sunlight.	9	3	Good response: complete chemical equation with reactants and products, mentions chloroplasts and chlorophyll, names the two stages. Bad response: one-sentence summary that is technically true but contains zero mechanism, no chemistry, no cellular detail — would score poorly in any evaluation rubric.
FQ-012	Factual Q&A	Who wrote the play Hamlet?	Hamlet was written by William Shakespeare, likely between 1599 and 1601. It is one of Shakespeare's most famous tragedies, centered on Prince Hamlet of Denmark and themes of revenge, mortality, and corruption. It is also one of the longest plays in the Shakespearean canon.	Hamlet was written by Shakespeare.	8	4	Good response: correct author with full name, approximate date, genre (tragedy), key themes, and an additional interesting fact. Bad response: technically correct but minimal — just author, no date, no context, no themes. For a simple factual question this might suffice, but provides no value beyond the bare fact.
FQ-013	Factual Q&A	What is the boiling point of water at sea level?	Water boils at 100°C (212°F) at standard sea-level atmospheric pressure (1 atm or 101.325 kPa). The boiling point decreases at higher altitudes due to lower atmospheric pressure, which is why cooking times must be adjusted at elevation.	Water boils at 100 degrees.	9	3	Good response: both temperature scales (°C and °F), specifies the condition (sea level, 1 atm), adds the altitude effect for completeness. Bad response: omits the unit (100 degrees what? — Celsius or Fahrenheit?), no mention of pressure dependency, ambiguous.
FQ-014	Factual Q&A	What is the difference between a virus and a bacterium?	Viruses and bacteria are both microscopic pathogens but differ fundamentally. Bacteria are single-celled living organisms with their own cellular machinery (ribosomes, cell membrane, DNA), and can reproduce independently. Viruses are not considered living organisms — they are particles of genetic material (DNA or RNA) wrapped in a protein coat, and can only replicate inside a host cell. Bacteria can be treated with antibiotics; viruses cannot (antivirals are used instead).	Viruses are smaller than bacteria. Both can make you sick but they are different types of germs.	9	2	Good response: covers cellular structure, reproduction, living vs non-living classification, treatment difference — a comprehensive comparison. Bad response: only mentions size, calls both 'germs' (informal/imprecise), no mechanistic distinction, no mention of treatment differences.
FQ-015	Factual Q&A	How far is the Moon from Earth?	The Moon's distance from Earth varies because its orbit is elliptical. On average, the Moon is about 384,400 km (238,855 miles) from Earth. At its closest point (perigee) it is approximately 356,500 km, and at its farthest (apogee) about 406,700 km. Light takes about 1.3 seconds to travel from the Moon to Earth.	The Moon is about 384,000 km away from Earth.	9	5	Good response: average distance in both km and miles, explains orbital ellipse, gives perigee/apogee values, adds light-travel time for perspective. Bad response: correct average (rounded) but presents it as a fixed number without noting the elliptical orbit — technically acceptable but lacks depth.
IF-001	Instruction Following	List exactly 5 fruits, each on its own line, with no numbering or bullet points.	Apple Mango Banana Strawberry Kiwi	Here are 5 fruits for you: 1. Apple 2. Mango 3. Banana 4. Strawberry 5. Kiwi I hope that helps!	10	2	Good response: exactly 5 fruits, each on its own line, no numbering, no bullets, no preamble. Follows every constraint. Bad response: adds numbering (explicitly prohibited), adds intro sentence and closing sentence (not requested), pads with unnecessary text — fails multiple explicit instructions.
IF-002	Instruction Following	Translate the following sentence into French, and then into Spanish. Do not add any explanation. Sentence: 'The cat is on the roof.'	Le chat est sur le toit. El gato está en el tejado.	Sure! Here is the translation: French: Le chat est sur le toit. This is a common French phrase. Spanish: El gato está en el techo.	10	3	Good response: French first, Spanish second, no extra text. Follows the 'no explanation' instruction exactly. Bad response: adds 'Sure!', labels each language, adds an unrequested comment about French, and uses 'el techo' (a valid 3 but less precise word for roof compared to 'el tejado'). Fails format instruction.
IF-003	Instruction Following	Write a haiku about rain. A haiku must follow the 5-7-5 syllable pattern. Summarize the following text in exactly 2 sentences.	Soft rain taps the glass (5) Puddles mirror silver skies (7) Earth drinks the quiet (5)	Raindrops fall slowly down Wetting all the leaves and ground Water everywhere	10	2	Good response: correct 5-7-5 pattern (verified: Soft-rain-taps-the-glass=5, Pud-dles-mir-ror-sil-ver-skies=7, Earth-drinks-the-qui-et=5), evocative imagery. Bad response: appears to follow a 6-7-5 pattern (Rain-drops-fall-slow-ly-down=6 syllables) — fails the core formal constraint of a haiku.
IF-004	Instruction Following	Text: 'The Amazon rainforest, often called the lungs of the Earth, produces around 20% of the world's oxygen. It spans over 5.5 million square kilometers across nine countries in South America. Home to 10% of all species on Earth, it faces severe threats from deforestation driven by agriculture, logging, and urban expansion.'	The Amazon rainforest spans 5.5 million square kilometers across nine South American countries, produces roughly 20% of the world's oxygen, and hosts 10% of all Earth's species. However, it faces mounting threats from deforestation driven by agriculture, logging, and urban development.	The Amazon is called the lungs of the Earth. It produces 20% of oxygen. It covers a large area across nine countries and has many species. But deforestation is a major problem.	10	2	Good response: exactly 2 sentences, covers all key points (size, oxygen, biodiversity, threats), well-written. Bad response: 4 sentences — directly violates the 'exactly 2 sentences' constraint. Despite containing accurate facts, the format instruction is failed entirely.
IF-005	Instruction Following	Give me three tips for better sleep. Format your response as a numbered list with bold tip titles.	1. "Maintain a consistent sleep schedule" – Go to bed and wake up at the same time every day, even on weekends, to regulate your circadian rhythm. 2. "Limit screen exposure before bed" – Blue light from phones and computers suppresses melatonin production; try to avoid screens for at least an hour before sleeping. 3. "Optimize your sleep environment" – Keep your bedroom cool (around 65–68°F / 18–20°C), dark, and quiet for optimal sleep conditions.	Here are some tips: First, you should try to sleep at a consistent time. Also, avoid your phone before bed because the light keeps you awake. Additionally, make sure your room is comfortable.	10	2	Good response: numbered list ✓, bold tip titles ✓, substantial detail for each tip. Bad response: prose format instead of numbered list, no bold titles, uses 'First/Also/Additionally' as connectors — completely fails the format instructions while containing loosely similar content.

ID	Category	Prompt	Good Response	Bad Response	Good Score	Bad Score	Annotation
IF-006	Instruction Following	Write a product description for a wireless keyboard. Keep it under 50 words.	Stay cable-free with our compact wireless keyboard. Featuring ultra-quiet keys, a 10-meter Bluetooth range, and a 6-month battery life. It pairs seamlessly with laptops, tablets, and smartphones. Slim, lightweight, and available in three colors — productively wherever you go.	Our wireless keyboard is an excellent choice for people who want to work without cables getting in the way. It connects via Bluetooth and has a long battery life. The keys are comfortable to type on, and the keyboard is compatible with many different devices including laptops, tablets, phones, and desktop computers. It comes in multiple colors and is very lightweight.	9	3	Good response: 46 words (under 50 ✓), highlights key features concisely, ends with a punchy tagline. Bad response: approximately 70 words — exceeds the 50-word limit. The constraint is explicit and the response fails it despite being well-written otherwise.
IF-007	Instruction Following	Act as a pessimistic financial advisor. Respond to: 'Should I invest in index funds?'	Sure, invest in index funds. Watch the market crash right after you put your money in. Markets go up — until they don't. You'll probably panic-sell at the bottom like everyone else, lock in your losses, and spend years recovering. Even if you hold on, inflation eats your returns, fees nibble the rest, and by retirement the rules will have changed. But sure — everyone else is doing it.	As a financial advisor, I recommend index funds as they offer diversification and low fees. Historically, the S&P 500 has returned about 10% annually. They are one of the most reliable long-term investment vehicles available.	9		Good response: fully adopts the pessimistic persona, cynical but plausible concerns (panic-selling, inflation, fees, rule changes), entertaining and on-character. Bad response: completely ignores the 'pessimistic' persona instruction — gives a standard optimistic advisor recommendation. Fails the core role-play constraint.
IF-008	Instruction Following	Rewrite the following sentence using passive voice: 'The chef cooked the meal.'	The meal was cooked by the chef.	The chef had cooked the meal beautifully.	10	1	Good response: correct passive construction (subject = 'the meal', auxiliary 'was', past participle 'cooked', agent 'by the chef'). Bad response: still active voice with added 'had' (past perfect) and 'beautifully' (not in original). Adds words not present in original and fails the passive voice transformation.
IF-009	Instruction Following	Create a one-week meal plan for a vegetarian. Present it as a table with columns: Day, Breakfast, Lunch, Dinner.	[Day Breakfast Lunch Dinner Monday Oatmeal with berries Lentil soup + whole grain bread Vegetable stir-fry with tofu Tuesday Avocado toast + eggs Chickpea salad Paneer tikka masala + rice Wednesday Greek yogurt + granola Caprese sandwich Mushroom pasta Thursday Smoothie bowl Black bean tacos Veggie burger + sweet potato fries Friday Whole wheat pancakes Tomato basil soup Spinach ricotta stuffed peppers Saturday Veggie omelette Falafel wrap Dal tadka + naan Sunday French toast + fruit Quinoa Buddha bowl Eggplant parmesan	Here is a vegetarian meal plan: Monday: Oatmeal for breakfast, salad for lunch, pasta for dinner. Tuesday: Toast for breakfast, soup for lunch, rice and vegetables for dinner. Wednesday: Yogurt for breakfast, sandwich for lunch, curry for dinner.	10	2	Good response: full table format ✓, all 7 days ✓, all 3 columns ✓, specific varied meals ✓. Bad response: prose/list format instead of a table (fails format instruction), only covers 3 of 7 days (incomplete), meals are vague ('salad', 'soup').
IF-010	Instruction Following	Write a formal apology letter from a company to a customer who received a damaged product. Do not use first-person singular ('I').	Dear Valued Customer, On behalf of [Company Name], we sincerely apologize for the condition in which your recent order arrived. Receiving a damaged product is not the experience we strive to provide, and we fully understand the frustration this has caused. Our team has been notified and is committed to making this right. A replacement will be dispatched within 2–3 business days at no additional cost, along with a 20% discount on your next purchase as a gesture of goodwill. We value your trust and appreciate your patience. Sincerely, [Company Name] Customer Care	Dear Customer, I am sorry to hear that your product arrived damaged. I want you to know that I take this very seriously and I will make sure you get a replacement as soon as possible. I apologize for the inconvenience.	10	1	Good response: formal tone ✓, no first-person singular ✓, uses 'we' throughout ✓, includes specific remediation (replacement + discount). Bad response: uses 'I' five times in three sentences — directly violates the explicit constraint. Also informal in structure and offers no concrete resolution details.
IF-011	Instruction Following	Give me a word that rhymes with 'orange'.	Strictly speaking, 'orange' has no perfect rhymes in standard English — it's famously one of the hardest words to rhyme. The closest near-rhyme is 'sporange' (a botanical term for a spore case). Some poets use 'door-hinge' as a forced slant rhyme.	A word that rhymes with orange is 'porridge'.	9	1	Good response: honest about the linguistic fact (no perfect rhyme), offers the best genuine answer ('sporange'), mentions the creative workaround ('door-hinge'). Bad response: 'porridge' does not rhyme with 'orange' — different vowel and consonant sounds. A hallucinated/incorrect answer to an impossible request.
IF-012	Instruction Following	Explain the concept of machine learning to a 10-year-old. Keep it simple.	Imagine teaching a dog new tricks. You show the dog what to do, reward it when it gets it right, and it learns over time. Machine learning works the same way — instead of a dog, it's a computer program. You give it lots of examples, tell it when it's right or wrong, and it learns to recognize patterns on its own. That's how your phone learns to recognize your face!	Machine learning is a subset of artificial intelligence that uses statistical algorithms to enable systems to learn from data. It involves training models on datasets to identify patterns and make predictions without being explicitly programmed for each task.	10	1	Good response: perfect analogy (dog training), age-appropriate language, relatable real-world example (face recognition on phone), zero jargon. Bad response: uses 'subset', 'statistical algorithms', 'models', 'datasets' — completely inappropriate for a 10-year-old. Ignores the 'keep it simple' instruction entirely.
IF-013	Instruction Following	List the steps to make a cup of tea. Use imperative verbs to start each step.	1. Boil water in a kettle. 2. Place a tea bag in a clean mug. 3. Pour the hot water over the tea bag. 4. Steep for 3–5 minutes, depending on desired strength. 5. Remove the tea bag and discard. 6. Add milk, sugar, or lemon to taste. 7. Stir and enjoy.	First, you should boil some water. Then you need to put a tea bag in a cup. After that, the water should be poured in and you wait for it to steep. You can then add milk or sugar.	10	3	Good response: every step starts with an imperative verb (Boil, Place, Pour, Steep, Remove, Add, Stir) ✓, clear numbered steps, specific detail. Bad response: uses 'you should', 'you need to', 'you can' — not imperative constructions. Also has passive voice ('should be poured') and no imperative verbs as instructed.
IF-014	Instruction Following	Respond only with a JSON object containing three keys: name, age, and occupation. Fill in fictional values.	<pre>{ "name": "Priya Sharma", "age": 34, "occupation": "Marine Biologist" }</pre>	<pre>{ "name": "John Doe", "age": 28, "occupation": "Engineer" }</pre>	10	3	Good response: raw JSON only, no preamble, no code fences, exactly three keys, valid JSON syntax. Bad response: adds intro sentence, wraps in markdown code fences, adds a closing sentence — 'respond ONLY with a JSON object' means no surrounding text. Fails the 'only' constraint.
IF-015	Instruction Following	Write a tweet (max 280 characters) announcing a new coffee shop opening. Include a hashtag.	We're OPEN! Sip & Stay Coffee Co. is now welcoming you at 12 Maple Street. Fresh beans, cozy vibes, and your new daily ritual. Come say hello! #SipAndStay	We are excited to announce the grand opening of our new coffee shop! We serve a wide variety of coffees, teas, and pastries in a warm and inviting atmosphere. Please come visit us at our location and enjoy our special opening week discounts. We can't wait to see you there! #CoffeeShop #GrandOpening #NewOpening	10	4	Good response: 159 characters ✓, includes hashtag ✓, engaging, punchy, has emoji and address detail. Bad response: 286 characters — exceeds 280-character Twitter limit by 6 characters. Technically fails the constraint despite being well-written. Also uses 3 hashtags (not specifically prohibited, but the tweet is too long regardless).
CO-001	Coding	Write a Python function that checks if a number is prime.	<pre>def is_prime(n): """Return True if n is a prime number, False otherwise.""" if n < 2: return False if n == 2: return True if n % 2 == 0: return False for i in range(3, int(n**0.5) + 1, 2): if n % i == 0: return False return True</pre>	<pre>def is_prime(n): for i in range(2, n): if n % i == 0: return False return True</pre>	9	4	Good response: handles edge cases (n<2, n=2, even numbers), uses efficient sqrt optimization (O(√n) complexity), includes docstring, skips even numbers in loop. Bad response: works for small inputs but is O(n) — checks every number up to n, no edge case handling (returns True for n=0 and n=1, which are not prime), no docstring.
CO-002	Coding	Debug this Python code: <pre>"""python def divide(a, b): return a / b result = divide(10, 0) print(result) """</pre>	<pre>"""python def divide(a, b): if b == 0: return None # or raise a custom error, or return float("inf") return a / b result = divide(10, 0) if result is None: print("Error: Cannot divide by zero.") else: print(result) """</pre> Alternatively, use a try/except block: <pre>"""python try: result = divide(10, 0) except ZeroDivisionError: print("Error: Division by zero.") """</pre>	The problem is that you can't divide by zero. Change the 0 to a different number to fix it.	9	1	Good response: correctly identifies ZeroDivisionError, provides two valid solutions (guard clause and try/except), shows complete runnable code. Bad response: identifies the problem but offers no code fix — 'change the 0 to a different number' is not debugging, it's changing the problem. Not useful to a developer.

ID	Category	Prompt	Good Response	Bad Response	Good Score	Bad Score	Annotation
CO-003	Coding	Write a JavaScript function that reverses a string.	<pre>'''javascript function reverseString(str) { return str.split('').reverse().join(''); } // Example: console.log(reverseString('hello')); // Output: 'olleh' '''</pre> This is a list comprehension that builds a list of squared values for even numbers from 0 to 9. Breaking it down: - <code>range(10)</code> generates numbers 0 through 9 - <code>'if x % 2 == 0'</code> filters to keep only even numbers (0, 2, 4, 6, 8) - <code>'x**2'</code> squares each filtered number Result: <code>[0, 4, 16, 36, 64]</code>	<pre>'''javascript function reverseString(str) { let reversed = ''; for (let i = 0; i <= str.length; i++) { reversed = str[i] + reversed; } return reversed; } '''</pre> This code creates a list using a for loop. It squares numbers and checks if they are even. The result is a list of even squares.	9	3	Good response: clean, idiomatic JS using <code>split/reverse/join</code> , includes example usage. Bad response: has an off-by-one error — loop condition is <code>'i <= str.length'</code> instead of <code>'i < str.length'</code> . At <code>i=str.length</code> , <code>str[i]</code> is <code>'undefined'</code> , so the result will have <code>'undefined'</code> prepended to the reversed string.
CO-004	Coding	Explain what this Python code does: <pre>'''python result = [x**2 for x in range(10) if x % 2 == 0] '''</pre>	<pre>def second_largest(lst): """Return the second largest unique value in a list.""" unique = list(set(lst)) if len(unique) < 2: return None # Not enough unique values unique.sort(reverse=True) return unique[1] # Test: print(second_largest([3, 1, 4, 1, 5, 9, 2, 6])) # Output: 6</pre> <code>'=='</code> is the loose equality operator — it compares values after type coercion. For example, <code>'5 == 5'</code> returns <code>'true'</code> because the string <code>'5'</code> is coerced to a number. <code>'==='</code> is the strict equality operator — it compares both value AND type with no coercion. So <code>'5 === 5'</code> returns <code>'false'</code> because one is a number and the other a string. Best practice: use <code>'==='</code> in most cases to avoid unexpected coercion bugs.	<pre>def second_largest(lst): lst.sort() return lst[-2]</pre>	10	4	Good response: names the construct (list comprehension), breaks down each component with code references, shows the actual output. Bad response: calls it a <code>'for loop'</code> (imprecise — it's a list comprehension), doesn't break down the components, doesn't show the actual result values.
CO-005	Coding	Write a Python function to find the second largest number in a list.	<pre>def flatten(lst): """Recursively flatten a nested list of any depth.""" result = [] for item in lst: if isinstance(item, list): result.extend(flatten(item)) else: result.append(item) return result # Test: print(flatten([[1, 2], [3, [4, 5]]])) # Output: [1, 2, 3, 4, 5]</pre> This uses a subquery to calculate the average salary across all employees, then filters the outer query to return only those earning above that average.	<pre>def second_largest(lst): lst.sort() return lst[-2]</pre>	9	3	Good response: handles duplicate values using <code>set()</code> , handles edge case of <code><2</code> unique values, includes docstring and test. Bad response: sorts and returns <code>lst[-2]</code> — fails for lists with duplicates (<code>[5, 5, 3]</code> would return 5 instead of 3), no edge case handling for empty or single-element lists, modifies the original list (side effect).
CO-006	Coding	What is the difference between <code>'=='</code> and <code>'==='</code> in JavaScript?	<pre>'''sql SELECT employee_id, name, salary FROM employees WHERE salary > (SELECT AVG(salary) FROM employees); '''</pre> This uses a subquery to calculate the average salary across all employees, then filters the outer query to return only those earning above that average.	<pre>'''sql SELECT * FROM employees WHERE salary > AVG(salary); '''</pre>	9	4	Good response: defines both operators, explains type coercion, gives a concrete example with output, adds best practice advice. Bad response: technically correct but gives no example, no explanation of type coercion mechanism, no practical guidance — insufficient for someone trying to understand when to use each.
CO-007	Coding	Write a SQL query to find all employees who earn more than the average salary.	<pre>def flatten(lst): """Recursively flatten a nested list of any depth.""" result = [] for item in lst: if isinstance(item, list): result.extend(flatten(item)) else: result.append(item) return result # Test: print(flatten([[1, 2], [3, [4, 5]]])) # Output: [1, 2, 3, 4, 5]</pre> <code>'async/await'</code> is syntactic sugar built on top of Promises that makes asynchronous code look and behave more like synchronous code. - <code>'async'</code> before a function declaration makes it return a Promise automatically. - <code>'await'</code> can only be used inside an <code>'async'</code> function; it pauses execution of that function until the awaited Promise resolves, then returns the resolved value. Example: <pre>'''javascript async function fetchData() { const response = await fetch('https://api.example.com/data'); const data = await response.json(); return data; } '''</pre> Without <code>async/await</code>, the same code would use <code>'.then()'</code> chains, which are harder to read.	<pre>def flatten(lst): flat = [] for sublist in lst: for item in sublist: flat.append(item) return flat</pre>	9	1	Good response: correct use of subquery for AVG, selects specific useful columns, includes explanation. Bad response: <code>'WHERE salary > AVG(salary)'</code> is invalid SQL — aggregate functions cannot be used directly in a <code>WHERE</code> clause. This query would throw a syntax/runtime error in any SQL engine.
CO-008	Coding	Write a Python function to flatten a nested list, e.g. <code>[[1,2],[3,[4,5]]] → [1,2,3,4,5]</code> .	<pre>def flatten(lst): """Recursively flatten a nested list of any depth.""" result = [] for item in lst: if isinstance(item, list): result.extend(flatten(item)) else: result.append(item) return result # Test: print(flatten([[1, 2], [3, [4, 5]]])) # Output: [1, 2, 3, 4, 5]</pre> <code>'async/await'</code> is syntactic sugar built on top of Promises that makes asynchronous code look and behave more like synchronous code. - <code>'async'</code> before a function declaration makes it return a Promise automatically. - <code>'await'</code> can only be used inside an <code>'async'</code> function; it pauses execution of that function until the awaited Promise resolves, then returns the resolved value. Example: <pre>'''javascript async function fetchData() { const response = await fetch('https://api.example.com/data'); const data = await response.json(); return data; } '''</pre> Without <code>async/await</code>, the same code would use <code>'.then()'</code> chains, which are harder to read.	<pre>def flatten(lst): flat = [] for sublist in lst: for item in sublist: flat.append(item) return flat</pre>	9	3	Good response: recursive approach handles arbitrary nesting depth, uses <code>isinstance</code> check, includes docstring and test matching the prompt's example. Bad response: only handles one level of nesting — the inner loop <code>'for item in sublist'</code> assumes each element of list is itself iterable, so <code>[[1,2],[3,[4,5]]]</code> would give <code>[1,2,3,[4,5]]</code> (fails to flatten the inner <code>[4,5]</code>).
CO-009	Coding	What does the <code>'async/await'</code> syntax do in JavaScript?	<pre>def word_frequency(text): """Return a dictionary of word frequencies from a string.""" words = text.lower().split() return dict(Counter(words)) # Test: print(word_frequency("the cat sat on the mat the cat")) # Output: {"the": 3, "cat": 2, "sat": 1, "on": 1, "mat": 1}</pre> A REST API (Representational State Transfer API) is a web interface that allows software systems to communicate over HTTP using standard methods: - <code>'GET'</code> — Retrieve data (e.g., <code>'GET /users/123'</code> fetches user 123) - <code>'POST'</code> — Create new data (e.g., <code>'POST /users'</code> creates a new user) - <code>'PUT/PATCH'</code> — Update existing data - <code>'DELETE'</code> — Remove data REST APIs are stateless — each request contains all the information needed; the server stores no session state. Responses are usually in JSON format. Example: when you open a weather app, it sends a <code>GET</code> request to a weather API and displays the JSON response.	<code>async/await</code> is a way to handle asynchronous code in JavaScript. It makes your code wait for something to finish before moving on.	9	3	Good response: mentions Promises (the foundation), explains both keywords separately, concrete code example, contrasts with <code>.then()</code> chains. Bad response: vague description, no mention of Promises, no code example, no explanation of what <code>'async'</code> does to a function's return value — not useful to a developer.
CO-010	Coding	Write a Python function that counts the frequency of each word in a string.	<pre>def word_frequency(text): """Return a dictionary of word frequencies from a string.""" words = text.lower().split() return dict(Counter(words)) # Test: print(word_frequency("the cat sat on the mat the cat")) # Output: {"the": 3, "cat": 2, "sat": 1, "on": 1, "mat": 1}</pre> A REST API (Representational State Transfer API) is a web interface that allows software systems to communicate over HTTP using standard methods: - <code>'GET'</code> — Retrieve data (e.g., <code>'GET /users/123'</code> fetches user 123) - <code>'POST'</code> — Create new data (e.g., <code>'POST /users'</code> creates a new user) - <code>'PUT/PATCH'</code> — Update existing data - <code>'DELETE'</code> — Remove data REST APIs are stateless — each request contains all the information needed; the server stores no session state. Responses are usually in JSON format. Example: when you open a weather app, it sends a <code>GET</code> request to a weather API and displays the JSON response.	<pre>def word_frequency(text): words = text.split() freq = {} for word in words: freq[word] = freq.get(word, 0) + 1 return freq</pre>	8	6	Good response: uses <code>Counter</code> for cleanliness, lowercases text to avoid case-sensitive duplicates (important edge case), includes test with expected output. Bad response: works correctly for basic input but doesn't lowercase (<code>'The'</code> and <code>'the'</code> would be counted separately), no test. Functional but misses an important edge case — score is moderate, not 1.
CO-011	Coding	What is a REST API and how does it work?	<pre>def word_frequency(text): """Return a dictionary of word frequencies from a string.""" words = text.lower().split() return dict(Counter(words)) # Test: print(word_frequency("the cat sat on the mat the cat")) # Output: {"the": 3, "cat": 2, "sat": 1, "on": 1, "mat": 1}</pre> A REST API (Representational State Transfer API) is a web interface that allows software systems to communicate over HTTP using standard methods: - <code>'GET'</code> — Retrieve data (e.g., <code>'GET /users/123'</code> fetches user 123) - <code>'POST'</code> — Create new data (e.g., <code>'POST /users'</code> creates a new user) - <code>'PUT/PATCH'</code> — Update existing data - <code>'DELETE'</code> — Remove data REST APIs are stateless — each request contains all the information needed; the server stores no session state. Responses are usually in JSON format. Example: when you open a weather app, it sends a <code>GET</code> request to a weather API and displays the JSON response.	A REST API is an API that uses the REST architecture. It allows different systems to communicate with each other over the internet using HTTP requests and JSON.	9	3	Good response: expands the acronym, lists all HTTP methods with examples, explains statelessness (a core REST principle), mentions JSON, gives a real-world analogy. Bad response: circular definition (<code>'API that uses REST architecture'</code>), mentions HTTP and JSON but no explanation of methods, statelessness, or any practical detail.

ID	Category	Prompt	Good Response	Bad Response	Good Score	Bad Score	Annotation
CO-012	Coding	Fix the bug in this code: <pre>python my_list = [1, 2, 3, 4, 5] for i in range(len(my_list)): if my_list[i] % 2 == 0: my_list.pop(i) print(my_list) ...</pre>	The bug: modifying a list while iterating over it by index causes skipped elements. When <code>my_list.pop(i)</code> removes an element, all subsequent elements shift left, so the next index misses an element — and can also cause an <code>IndexError</code>. Fixed version: <pre>python my_list = [1, 2, 3, 4, 5] my_list = [x for x in my_list if x % 2 != 0] print(my_list) # Output: [1, 3, 5] ...</pre> Alternative using filter: <pre>python my_list = list(filter(lambda x: x % 2 != 0, my_list)) ...</pre>	The code looks correct to me. It removes even numbers from the list.	10	1	Good response: correctly identifies the classic 'mutating a list during iteration' bug, explains why it fails, gives two correct alternatives with output. Bad response: completely fails to identify the bug — says the code 'looks correct'. This is a dangerous response that would mislead a developer.
CO-013	Coding	Write a Python decorator that logs the execution time of a function.	<pre>import time import functools def timer(func): """Decorator that logs execution time of a function.""" @functools.wraps(func) def wrapper(*args, **kwargs): start = time.perf_counter() result = func(*args, **kwargs) end = time.perf_counter() print(f"{func.__name__} executed in {end - start:.4f} seconds") return result return wrapper @timer def slow_function(): time.sleep(1) slow_function() # Prints: 'slow_function' executed in 1.0012 seconds</pre>	<pre>def timer(func): def wrapper(): import time start = time.time() func() print("Time:", time.time() - start) return wrapper</pre>	9	4	Good response: uses <code>functools.wraps</code> (preserves metadata), uses <code>perf_counter</code> (more precise than <code>time()</code>), handles <code>*args/**kwargs</code> , returns result, formats output nicely, includes usage example. Bad response: doesn't use <code>functools.wraps</code> , doesn't pass <code>*args/**kwargs</code> (can't decorate functions with arguments), doesn't return the function's result, uses less precise <code>time.time()</code> .
CO-014	Coding	What is the time complexity of binary search and why?	Binary search has a time complexity of $O(\log n)$. Here's why: at each step, binary search eliminates half of the remaining search space by comparing the target to the middle element. Starting with n elements: - After 1 comparison: $n/2$ remain - After 2 comparisons: $n/4$ remain - After k comparisons: $n/2^k$ remain The search ends when 1 element remains: $n/2^k = 1$, so $k = \log_2(n)$. Thus, at most $\log_2(n)$ comparisons are needed. For 1,000,000 elements, that's only about 20 comparisons.	Binary search is $O(\log n)$ because it keeps dividing the list in half.	9	4	Good response: states complexity, derives it mathematically, shows the halving pattern step by step, gives an intuitive real-number example (1M elements → 20 comparisons). Bad response: states the correct answer but gives only a one-clause explanation — 'Divides in half' without the mathematical derivation showing why that yields $\log n$.
CO-015	Coding	Write a function in Python to check whether a string is a palindrome.	<pre>def is_palindrome(s): """Return True if s is a palindrome (ignoring case and spaces).""" cleaned = s.lower().replace(" ", "") return cleaned == cleaned[::-1] # Tests: print(is_palindrome("racecar")) # True print(is_palindrome("A man a plan a canal Panama")) # True print(is_palindrome("hello")) # False</pre>	<pre>def is_palindrome(s): return s == s[::-1]</pre>	9	5	Good response: handles case insensitivity and spaces, includes doctest and multiple tests including a phrase palindrome. Bad response: works for simple cases ('racecar') but fails for 'A man a plan a canal Panama' (returns False due to spaces and case). Technically correct for simple inputs but incomplete.
RE-001	Reasoning	If all roses are flowers and some flowers fade quickly, can we conclude that some roses fade quickly?	No, we cannot logically conclude that some roses fade quickly. This is a syllogistic fallacy. The logic: 'All roses are flowers' and 'some flowers fade quickly' — but the flowers that fade quickly might be a subset of flowers that does not include any roses. Without knowing that any rose is in the 'fade quickly' group, we cannot make that inference. The two sets (roses and fast-fading flowers) may be entirely disjoint within the set of all flowers.	Yes, since all roses are flowers and some flowers fade quickly, it follows that some roses must fade quickly.	10	1	Good response: correctly identifies the logical fallacy, explains why the inference fails (the fading flowers could exclude all roses), uses clear set logic. Bad response: commits the fallacy — this is a classic example of invalid syllogism. 'Some A is B' + 'All C is A' does not imply 'Some C is B'.
RE-002	Reasoning	A bat and a ball cost \$1.10 in total. The bat costs \$1.00 more than the ball. How much does the ball cost?	The ball costs \$0.05 (5 cents). Here's the algebra: Let ball = x, bat = $x + \\$1.00$ Total: $x + (x + 1.00) = 1.10$ $2x + 1.00 = 1.10$ $2x = 0.10$ $x = 0.05$ The intuitive (wrong) answer is \$0.10, but if the ball were \$0.10, the bat would be \$1.10, making the total \$1.20 — not \$1.10.	The ball costs \$0.10, and the bat costs \$1.00.	10	1	Good response: correct answer (\$0.05), shows algebraic derivation, explicitly addresses and refutes the common intuitive wrong answer (\$0.10). Bad response: gives the classic intuitive wrong answer — $\$0.10 + \$1.00 = \$1.10$, but the 1 bat (\$1.00) is NOT \$1.00 more than the ball (\$0.10) — it's only \$0.90 more.
RE-003	Reasoning	If it takes 5 machines 5 minutes to make 5 widgets, how long does it take 100 machines to make 100 widgets?	Each machine makes 1 widget in 5 minutes (5 machines × 1 widget each = 5 widgets in 5 minutes). So 100 machines each make 1 widget in the same 5 minutes, producing 100 widgets total. The number of machines scales with the number of widgets — the time stays constant.	It would take 100 minutes, because 100 widgets would take 100 times longer than making 1 widget.	10	1	Good response: correct answer (5 minutes), explains the rate per machine, shows why the time doesn't scale. Bad response: wrong answer (100 minutes) — makes an incorrect proportional reasoning error, confusing the number of widgets with time when machines increase proportionally.
RE-004	Reasoning	In a room, there are 3 light switches outside, each controlling one of 3 bulbs inside. You can only enter the room once. How do you identify which switch controls which bulb?	Turn switch 1 ON for several minutes, then turn it OFF. Turn switch 2 ON. Enter the room: - The bulb that is ON → controlled by switch 2 - The bulb that is OFF but warm → controlled by switch 1 (it was on long enough to heat up) - The bulb that is OFF and cold → controlled by switch 3 This works because light bulbs retain heat after being switched off, giving you a third distinguishable state beyond just on/off.	Turn on switch 1, enter the room, see which bulb is on. That's switch 1. Then you go back out and turn on switch 2 and enter again.	10	1	Good response: uses the heat property of bulbs to create three distinguishable states (on, off-warm, off-cold), solves within the one-entry constraint. Bad response: proposes entering the room multiple times — directly violates the 'only 1 enter once' constraint. Not a valid solution.
RE-005	Reasoning	You have a 3-liter jug and a 5-liter jug and unlimited water. How do you measure exactly 4 liters?	Step 1: Fill the 5-liter jug completely. Step 2: Pour from the 5-liter into the 3-liter until the 3-liter is full. (5-liter now has 2 liters.) Step 3: Empty the 3-liter jug. Step 4: Pour the 2 liters from the 5-liter into the 3-liter jug. Step 5: Fill the 5-liter jug completely. Step 6: Pour from the 5-liter into the 3-liter until it's full (it needs 1 more liter). Result: The 5-liter jug now contains exactly 4 liters.	Fill the 3-liter jug and pour it into the 5-liter jug twice. That gives you 6 liters. Then pour out 2 liters to get 4 liters.	10	1	Good response: valid 6-step solution using only the two jugs, each step is measurable with the available containers. Bad response: 'pour out exactly 2 liters' from the 5-liter jug — but you have no way to measure 2 liters exactly without the 1 problem being circular. The solution assumes a measurement ability that doesn't exist.

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]